Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

# ЗВІТ

з лабораторної роботи № 9
з дисципліни «Основи програмування»
**«Стрічки»**

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

## Мета роботи

Вивчити поняття і застосування стрічок.

## 1. Завдання 1. Реверс слів у прислів'ї

Програма читає рядок (прислів'я), розбиває його на слова і для кожного слова змінює порядок символів на зворотний. Використано широкі символи wchar_t, щоб коректно працювати з кирилицею.

### 1.1. Як влаштовано код

Підключено string.h, wchar.h, locale.h. Функція reverse_word міняє місцями символи з початку і з кінця рядка, поки індекси не зустрінуться посередині. У main спочатку setlocale(LC_ALL, ""), щоб fgetws і wprintf розуміли українські літери. Рядок читається через fgetws, далі wcstok ріже його по пробілах у масив words. Кожне слово передається в reverse_word і друкується.



```c
#include <stdio.h>
#include <string.h>
#include <wchar.h>
#include <locale.h>

#define MAX_WORDS 50
#define MAX_WORD_LEN 100
#define MAX_LINE_LEN 500

void reverse_word(wchar_t *str) {
    int len = wcslen(str);
    wchar_t temp;
    for (int i = 0; i < len / 2; i++)
    {
        temp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = temp;
    }
}



int main(void) {
    setlocale(LC_ALL, "");
    wchar_t line[MAX_LINE_LEN];
    wchar_t words[MAX_WORDS][MAX_WORD_LEN];
    int word_count = 0;

    wprintf(L"Введіть прислів'я: ");
    if (fgetws(line, MAX_LINE_LEN, stdin) == NULL) {
        return 1;
    }

    wchar_t *temp;
    wchar_t *token = wcstok(line, L" ", &temp);
    while (token != NULL && word_count < MAX_WORDS) {
        wcscpy(words[word_count], token);
        words[word_count][MAX_WORD_LEN - 1] = '\0';
        word_count++;
        token = wcstok(NULL, L" ", &temp);
    }
    for (int i = 0; i < word_count; i++) {
        reverse_word(words[i]);
        wprintf(L"%ls\n", words[i]);
    }

    return 0;
}
```

Рис. 1. Код task1.c — reverse_word та розбір рядка через wcstok

### 1.2. Результат виконання

На вхід подано: «Хто не був високо, той зроду не збагне, як страшно впасти». Кожне слово виведено задом наперед: отХ, ен, вуб, … — тобто «Хто» стало «отХ», «не» → «ен» і так далі. Пробіли між словами збережено як окремі токени; розділові знаки, якщо вони були приліплені до слова, їдуть разом із ним.

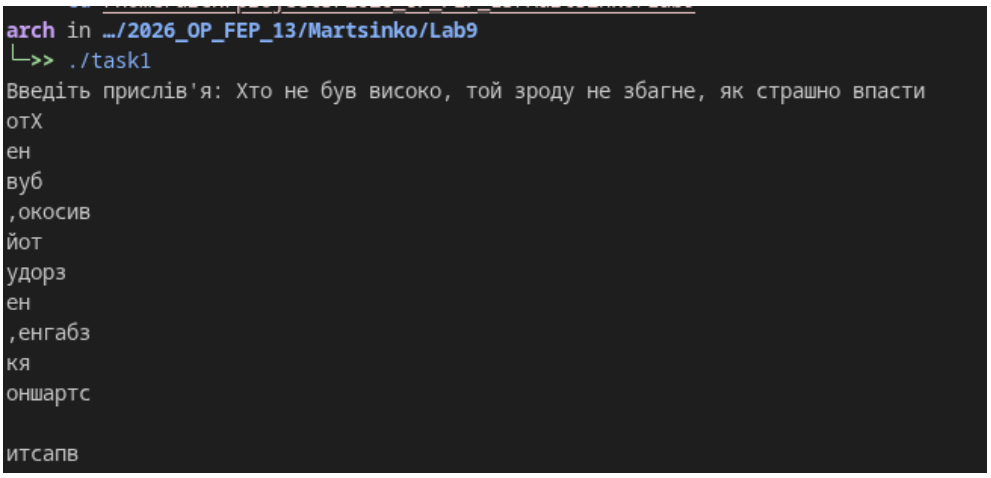



Рис. 2. Вивід task1: кожне слово прислів'я у зворотному порядку

## 2. Завдання 2. Список групи: сортування, унікальні імена, середня довжина прізвищ

Програма читає список групи ентів (номер, прізвище, ім'я), сортує за іменем, а при однаковому імені — за прізвищем, рахує скільки унікальних імен і середню довжину прізвищ.

### 2.1. Як влаштовано код

Оголошено структуру group з полями id, surname, name (wchar_t). Функція compare для qsort спочатку порівнює імена через wcscoll (з урахуванням української локалі), якщо імена рівні — прізвища. У main рядки читаються fgetws, парсяться swscanf у структуру, накопичуються в масив. Після qsort список друкується. Унікальні імена рахуються одним проходом по вже відсортованому масиву: якщо поточне ім'я відрізняється від попереднього — лічильник +1. Середня довжина прізвищ — сума wcslen усіх surname, поділена на кількість записів.

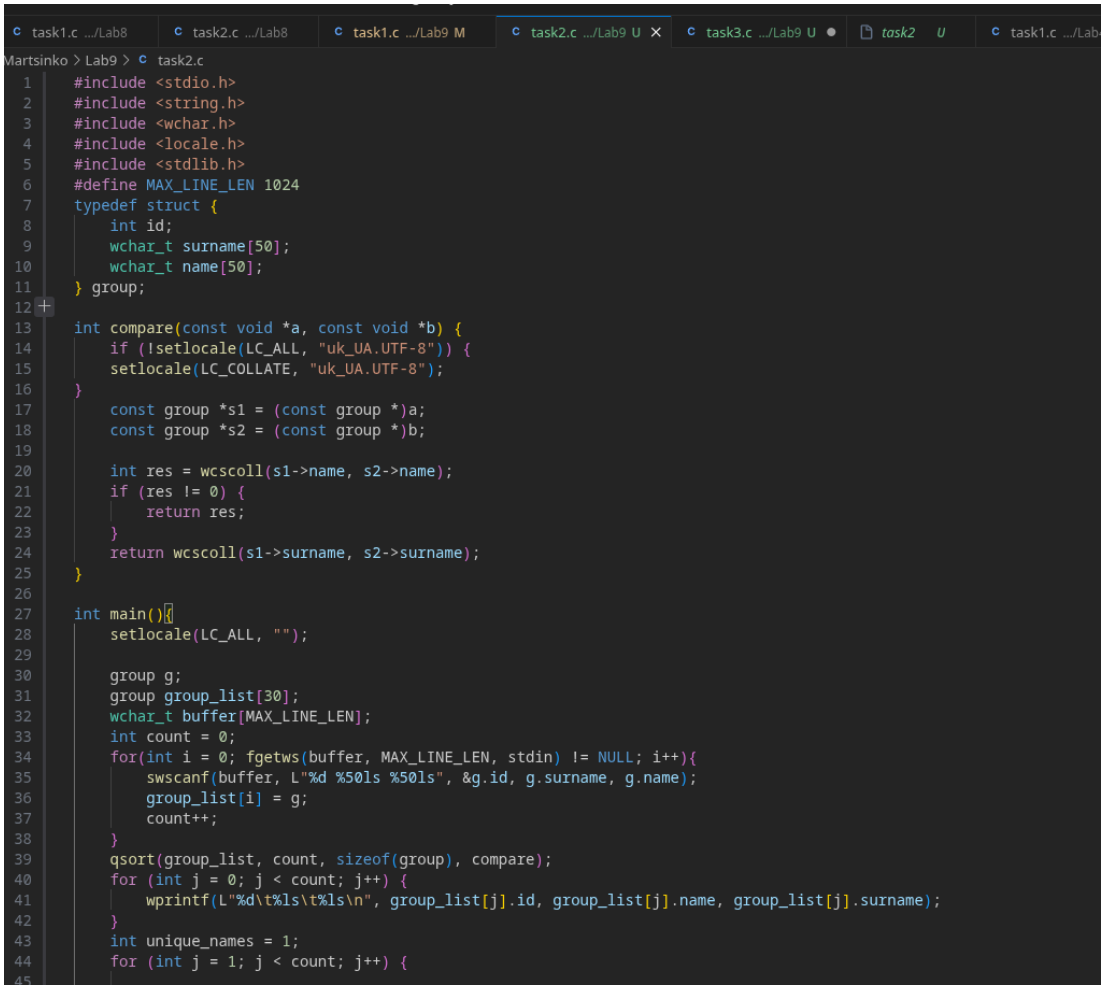



Рис. 3. Код task2.c — структура group, compare та qsort

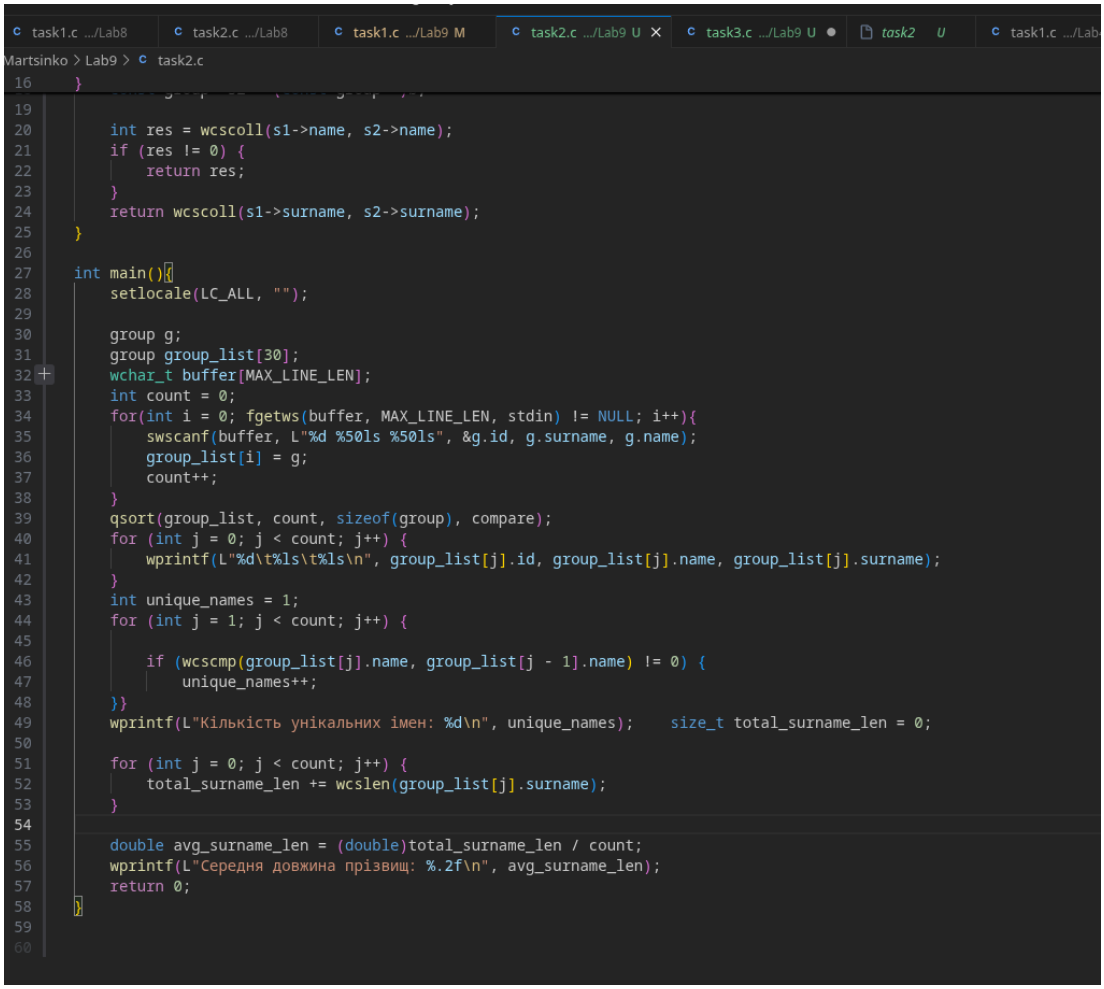



Рис. 4. Код task2.c — підрахунок унікальних імен і середньої довжини прізвищ

## 2.2. Результат виконання

Спочатку виводиться вихідний список (номер, прізвище, ім'я). Після сортування — той самий список, але впорядкований за іменами (Анатолій, Андрій, Валерій…). У кінці: «Кількість унікальних імен: 21» і «Середня довжина прізвищ: 7.88». Сортування відпрацювало коректно для кирилиці завдяки wcscoll і локалі uk_UA.

```
—>> ./task2
1 Єрмаков Георгій
2 Крупський Ілля
3 Сарапій Едуард
4 Михавко Тарас
5 Бондар Валерій
6 Бражко Володимир
7 Ярмоленко Андрій
8 Назаренко Олександр
9 Яремчук Роман
10 Пономаренко Матвій
11 Ванат Владислав
12 Трубін Анатолій
13 Забарний Ілля
14 Драмбаєв Олександр
15 Циганков Віктор
16 Варфоломєєв Іван
16 Миколенко Віталій
17 Піхальонок Олександр
18 Очеретько Олег
19 Назарина Єгор
20 Хлань Максим
21 Велетень Владислав
22 Краснопір Ігор
22 Матвієнко Микола
23 Різник Дмитро
12      Анатолій        Трубін
7       Андрій  Ярмоленко
5       Валерій Бондар
15      Віктор  Циганков
16      Віталій Миколенко
11      Владислав       Ванат
21      Владислав       Велетень
6       Володимир       Бражко
1       Георгій Єрмаков
23      Дмитро  Різник
3       Едуард  Сарапій
19      Єгор    Назарина
```

Рис. 5. Вихідний список групи та початок відсортованого

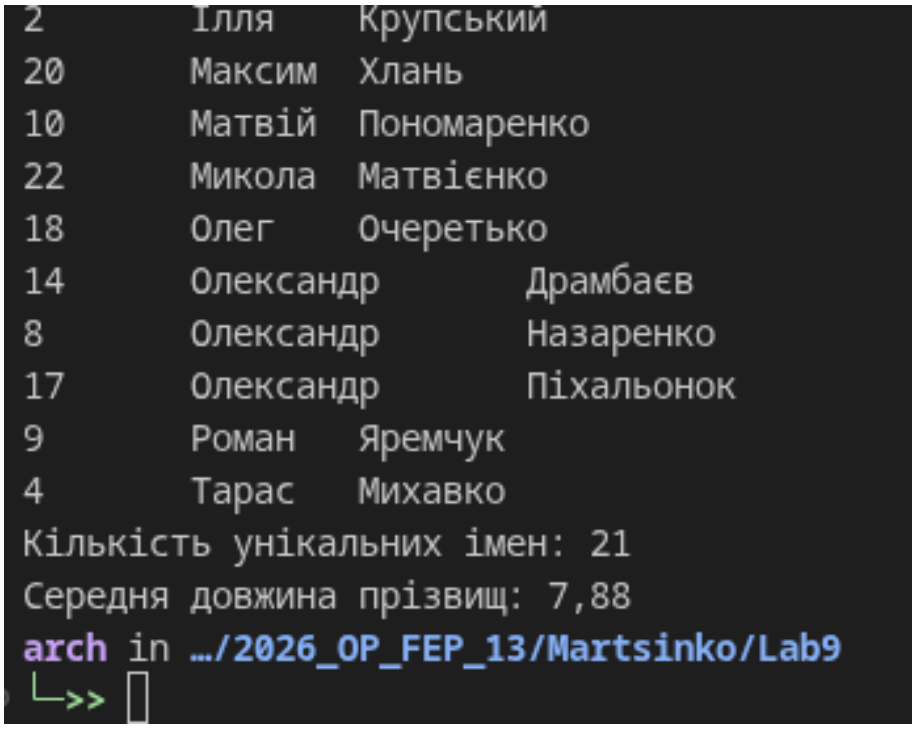



Рис. 6. Кінець відсортованого списку, унікальні імена та середня довжина прізвищ

## 3. Завдання 3. Перевірка email регулярним виразом

Програма читає рядок і перевіряє, чи він схожий на коректну email-адресу, за допомогою POSIX regex (regex.h).

### 3.1. Як влаштовано код

Шаблон: локальна частина (літери, цифри, ._%+-), символ @, домен з крапками, опційний додатковий рівень домену. regcomp компілює вираз з прапорцями REG_EXTENDED | REG_NOSUB. regexec перевіряє введений рядок; якщо повернуло 0 — збіг («Правильний»), інакше «Неправильний». Перед перевіркою з рядка прибирається символ кінця рядка.

Рис. 7. Код task3.c — шаблон regex і перевірка email

Martsinko > Lab9 > C task3.c

```c
#include <stdio.h>
#include <string.h>
#include <regex.h>

#define MAX_LEN 256

int main(void) {
    char email[MAX_LEN];
    regex_t regex;
    int reti;

    const char *pattern = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9-]+\\.[a-zA-Z0-9-]+(\\.[a-zA-Z0-9-]+)?$";

    printf("Введіть email: ");

    if (fgets(email, sizeof(email), stdin) == NULL) {
        return 0;
    }

    email[strcspn(email, "\r\n")] = '\0';

    if (regcomp(&regex, pattern, REG_EXTENDED | REG_NOSUB) != 0) {
        fprintf(stderr, "Помилка компіляції regex\n");
        return 1;
    }

    reti = regexec(&regex, email, 0, NULL, 0);
    regfree(&regex);

    if (reti == 0) {
        printf("Правильний\n");
    } else {
        printf("Неправильний\n");
    }

    return 0;
}
```

Рис. 8. Код task3.c (повний вигляд)

### 3.2. Результат виконання

Введено nazar1404m@gmail.com — програма відповіла «Правильний». Шаблон прийняв типову адресу з латинськими літерами, цифрами, @ і доменом другого рівня.

```
arch in …/2026_OP_FEP_13/Martsinko/Lab9
└─>> ./task3
Введіть email: nazar1404m@gmail.com
Правильний
```

Рис. 9. Вивід task3: email розпізнано як правильний

### Висновок

У цій роботі розібрано три типові сценарії зі стрічками в C. У першому завданні головне — не забути локаль і wchar_t: без setlocale кирилиця в fgetws просто не читається нормально, а звичайний char ламає українські літери. Реверс слова через обмін символів з кінців — простий і зрозумілий прийом, без зайвих буферів.

У другому завданні зручно винести студента в структуру і віддати порівняння в qsort: wcscoll з українською локаллю сортує імена так, як очікуєш по алфавіту, а не «як вийде» по кодах символів. Підрахунок унікальних імен після сортування —

один лінійний прохід, без вкладених циклів. Середня довжина прізвищ через wcslen теж без сюрпризів.

Третє завдання показало, що для перевірки формату email зручніше не писати десятки if вручну, а один раз скласти регулярний вираз. regex.h у C трохи багатослівний (regcomp / regexec / regfree), але для навчальної перевірки шаблону цього достатньо. Усі три програми зібрано і перевірено на реальних вхідних даних.